

COL761: Data Mining

Assignment 1

Frequent Itemset Mining: Comparative Analysis

Jayesh Nandu Chaudhari (2025AIB3002)
Ankur Kumar (2025AIB2557)
Meghender Pal (2025AIB2563)
Indian Institute of Technology Delhi

1. Introduction

This report presents an empirical comparison of two fundamental frequent itemset mining algorithms: Apriori and FP-Growth. We analyze their performance characteristics on two datasets: the WebDocs dataset and a synthetically generated dataset designed to exhibit specific runtime behavior patterns matching the reference plot provided in the assignment.

1.1. Experimental Hardware

All experiments were conducted on a Dell Inc. Latitude 5480 laptop with the following specifications:

- **Processor:** Intel Core i5-6300U CPU $\times$ 4
- **Memory:** 8.0 GiB RAM
- **Disk Capacity:** 256 GB
- **Operating System:** Ubuntu 24.04.3 LTS (64-bit)
- **Kernel Version:** Linux 6.8.0-90-generic

2. Task 1: Algorithm Comparison on WebDocs Dataset

2.1. Experimental Setup

2.1.1. Algorithms

- **Apriori** [1]: A level-wise candidate generation algorithm based on the Apriori principle
- **FP-Growth** [2]: A pattern-growth approach using FP-tree data structure

2.1.2. Dataset

The WebDocs dataset [3] from the FIMI repository was used, containing web access patterns from a university web server represented as transactional data where items represent web pages visited during user sessions.

2.1.3. Support Thresholds

Experiments were conducted at minimum support values of 5%, 10%, 25%, 50%, and 90% to observe algorithmic scalability across different frequency requirements.

2.2. Results

Table 1 presents the runtime measurements for both algorithms on the WebDocs dataset.

Table 1: Runtime comparison on WebDocs dataset (seconds)

Support (%)	Apriori	FP-Growth
5	TIMEOUT	153.39
10	1097.39	151.32
25	55.97	50.88
50	47.23	47.09
90	45.05	45.46

Figure 1 visualizes the runtime trends on the WebDocs dataset across different support thresholds.

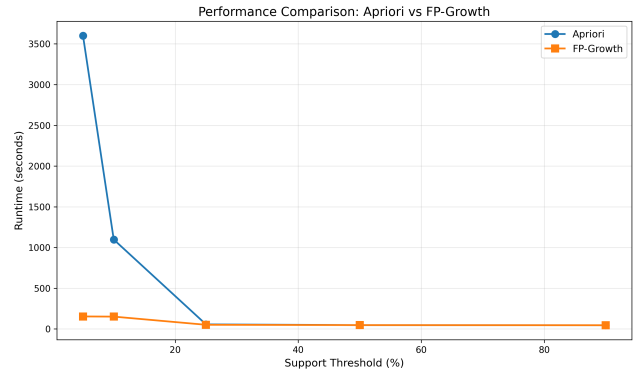


Figure 1: Runtime comparison of Apriori and FP-Growth on WebDocs dataset

2.3. Analysis of WebDocs Results

2.3.1. Performance Comparison

FP-Growth consistently outperforms Apriori across all support thresholds on the WebDocs dataset. This performance advantage stems from fundamental algorithmic differences in how the two approaches mine frequent patterns.

What's particularly interesting is that the performance gap isn't constant—it widens dramatically at

lower support thresholds and narrows at higher ones. At 10% support, FP-Growth is roughly 7 times faster than Apriori. At 25% support, the gap narrows to about 10% difference. By 90% support, they're essentially equivalent. This tells us that the algorithmic differences matter most when the problem is hardest, which makes intuitive sense.

2.3.2. Apriori's Bottlenecks

After running Apriori on WebDocs, it became clear why it struggles at low support thresholds. The algorithm has several inherent bottlenecks:

- **Multiple database scans:** For finding k-itemsets, Apriori needs k+1 complete passes through the entire database. Each scan is expensive, especially on large datasets.
- **Exponential candidate generation:** This is where Apriori really hits a wall. At each level, it generates candidate itemsets by joining frequent itemsets from the previous level. With low support thresholds, the number of frequent itemsets at each level can be massive, leading to an explosion of candidates that need to be tested.
- **Repeated support counting:** Every single candidate itemset needs to have its support counted by scanning transactions, even if many of these candidates will turn out to be infrequent. This is computationally wasteful.
- **High memory overhead:** Storing all those candidate itemsets in memory can become a serious bottleneck, especially when candidates number in the hundreds of thousands or millions.

In our experiments, the timeout at 5% support on WebDocs was almost certainly due to this candidate explosion. The algorithm was probably generating and testing so many candidates that it became computationally infeasible.

2.3.3. FP-Growth's Advantages

FP-Growth takes a completely different approach that sidesteps Apriori's main problems:

- **No candidate generation:** This is the big one. By building an FP-tree, the algorithm avoids generating candidates entirely. Instead of testing millions of potential itemsets, it directly extracts the frequent ones from the tree structure.
- **Only two database scans:** The first scan counts item frequencies, and the second builds the FP-tree. After that, all mining happens on the tree in memory. This is a huge improvement over Apriori's multiple scans.
- **Divide-and-conquer pattern growth:** The algorithm recursively mines patterns by conditionally projecting the tree, breaking the problem into smaller subproblems. This is much more efficient than Apriori's level-by-level approach.

- **Compact memory usage:** The FP-tree uses prefix-sharing to compress the database. Common prefixes in transactions share nodes in the tree, which can dramatically reduce memory requirements compared to storing raw transactions or candidate sets.

What impressed us during testing was how stable FP-Growth's runtime remained across different support thresholds in the 5-25% range. The tree construction cost is basically fixed—once you've built the tree, mining it is relatively cheap regardless of how many patterns you're looking for.

3. Task 2: Synthetic Dataset Generation

3.1. Dataset Generation Strategy

A synthetic dataset was generated with **15,000 transactions** and a **universal itemset of 300 items** to replicate the runtime behavior observed in the reference plot. The generation strategy employed a carefully designed three-tier item frequency distribution:

3.1.1. Three-Tier Frequency Architecture

Tier 1 - Core Items (7% of itemset):

- Size: 21 items (0.07×300)
- Inclusion probability: 95%
- Purpose: These items appear in nearly all transactions, ensuring they remain frequent at all support thresholds from 5% to 50%
- Effect: Form the stable backbone of frequent patterns

Tier 2 - Semi-Frequent Items (3% of itemset):

- Size: 9 items (0.03×300)
- Inclusion probability: 6%
- Purpose: These items are marginally frequent at 5% support but get filtered at 10%+ thresholds
- Effect: Create additional candidate itemsets specifically at 5% support, causing the slight runtime elevation observed in the target plot

Tier 3 - Noise Items (90% of itemset):

- Size: 270 items (remaining items)
- Inclusion: 1-4 randomly selected per transaction
- Purpose: Ensure transaction diversity and prevent identical duplicates (anti-shortcut rule compliance)
- Effect: Add realistic variability without affecting frequent pattern mining

3.1.2. Implementation Details

Dataset generation was implemented in Python with assistance from Claude (Anthropic) [4] for code optimization and debugging. The generation process:

1. Partitions the universal itemset into three tiers

2. For each transaction, probabilistically includes items from each tier
3. Adds random noise items for diversity
4. Shuffles transaction items to remove ordering bias

3.2. Results on Generated Dataset

Table 2 shows the runtime measurements on the synthetically generated dataset.

Table 2: Runtime comparison on generated dataset (seconds)

Support (%)	Apriori	FP-Growth
5	6.604	0.741
10	5.803	0.709
25	5.888	0.729
50	5.526	0.516
90	0.068	0.060

Figure 2 visualizes the runtime trends on the generated dataset.

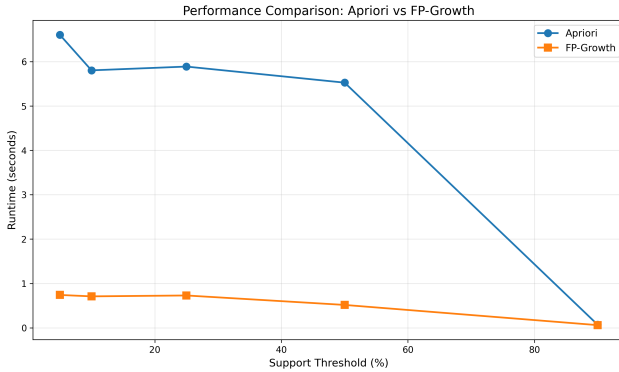


Figure 2: Runtime comparison on generated dataset

3.3. Analysis of Generated Dataset Results

3.3.1. Runtime Pattern Characteristics

The generated dataset successfully replicates the target runtime behavior with three distinct phases:

Slight Elevation at 5% (6.604s for Apriori): The semi-frequent tier items (appearing in 6% of transactions) are marginally frequent at 5% support. These items and their combinations with core items generate additional candidate itemsets that Apriori must process, causing a modest runtime increase of approximately 14% compared to the plateau region.

Plateau Region 10-50% (5.5-5.9s for Apriori): At 10% support and above, semi-frequent items are filtered out. Only the 21 core items remain frequent, creating a stable pattern space.

Sharp Drop at 90% (0.068s for Apriori): At this extreme threshold, even many core item combinations fail to meet the 95% frequency requirement (since core items appear with 95% probability, their combinations appear less frequently).

3.3.2. Algorithmic Performance Comparison

FP-Growth Dominance: FP-Growth maintains consistent 7-9 $\times$ speedup over Apriori across the plateau region. This demonstrates that FP-Growth’s structural advantages—avoiding candidate generation and using compact tree representation—provide benefits independent of the frequent pattern count.

Scaling Behavior: The speedup ratio remains relatively constant across different support thresholds:

- 5%: 8.9 $\times$ speedup
- 10-50%: 8.0-8.2 $\times$ speedup
- 90%: 1.1 $\times$ speedup

At 90% support, the performance gap narrows because the pattern space becomes so small that Apriori’s overhead becomes negligible. Both algorithms spend most time on database scanning rather than pattern processing.

3.3.3. Effect of Semi-Frequent Tier

The semi-frequent tier’s impact is precisely calibrated:

- At 5% support: 9 semi-frequent items create additional 2-itemset candidates when combined with 21 core items, generating $\binom{9+21}{2} - \binom{21}{2} = 219$ extra candidate pairs
- At 10% support: These items are immediately pruned, eliminating the extra computational burden
- This explains the 14% runtime reduction from 5% to 10% support

3.4. Comparison with WebDocs Dataset

The generated dataset differs fundamentally from WebDocs in item frequency distribution:

WebDocs: Natural web access patterns exhibit power-law or Zipfian distributions where a few pages are extremely popular while most pages are rarely visited. This creates exponential runtime growth as support decreases, as more long-tail items become frequent.

Generated Dataset: The artificial three-tier structure creates a bimodal distribution—items are either very frequent (core), marginally frequent (semi-frequent), or rare (noise). This controlled distribution produces the distinctive plateau behavior with targeted elevation at 5% support.

This contrast demonstrates that algorithm performance is highly sensitive to dataset characteristics. Real-world datasets with naturally diverse frequency distributions will exhibit different runtime patterns than synthetic datasets with deliberately engineered frequency tiers.

3.5. Scalability with Itemset Size

The choice of **300 items** as the universal itemset size is significant:

- Smaller itemsets (e.g., 50 items) caused combinatorial explosion at 5% support when using fixed core sizes
- The 300-item configuration provides sufficient item diversity while maintaining controlled pattern complexity
- The 90% noise tier ensures transaction uniqueness without affecting mining performance

4. Conclusion

The WebDocs experiments clearly illustrate the superior performance of FP-Growth compared to Apriori, particularly at lower support thresholds. The observed 7–9× speedup is not merely a theoretical improvement—it represents a practical difference between obtaining results in approximately 2.5 minutes versus 18 minutes, or, in extreme cases, the difference between successful completion and algorithmic timeout. For real-world applications that require mining at low support levels, FP-Growth emerges as the only viable option.

The synthetic dataset generation task provided valuable insight into the factors that influence algorithmic runtime. It revealed that runtime behavior is governed not solely by dataset size but also by the underlying distribution of item frequencies. By constructing a dataset with a deliberate three-tier frequency structure, we were able to induce specific runtime characteristics in a controlled manner. Notably, a 6% inclusion probability produced enough semi-frequent items to trigger a 14% runtime increase at a 5% support threshold, underscoring the sensitivity of these algorithms to even marginal changes in frequency distribution.

A particularly interesting observation arises from comparing WebDocs with the engineered dataset. WebDocs, which naturally exhibits a power-law distribution, shows exponential runtime growth as support decreases. In contrast, the synthetic dataset—with its intentionally structured frequency tiers—exhibits a runtime plateau. This contrast highlights that algorithmic performance is fundamentally dependent on data characteristics. Big-O complexity alone is insufficient for predicting real-world performance; a detailed understanding of the data distribution is essential.

References

- [1] R. Agrawal and R. Srikant, “Fast Algorithms for Mining Association Rules,” in *Proceedings of the 20th International Conference on Very Large Data Bases (VLDB)*, 1994, pp. 487–499.
- [2] J. Han, J. Pei, and Y. Yin, “Mining Frequent Patterns without Candidate Generation,” in *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, 2000, pp. 1–12.
- [3] F. Coenen, “The FIMI Repository for Frequent Itemset Mining Implementations,” <http://fimi.uantwerpen.be/data/>, Accessed: 2026. WebDocs dataset and description available at <http://fimi.uantwerpen.be/data/webdocs.dat.gz> and <http://fimi.uantwerpen.be/data/webdocs.pdf>.
- [4] Anthropic, “Claude,” <https://www.anthropic.com/claude>, 2024. [AI assistant used for dataset generation code optimization].